

Bayesian Probability Theory & Variational Autoencoders

From Statistical Foundations to Deep Generative Modelling on CIFAR-10

MarinKlm · HAI Project · 2026

CIFAR-10	latent dim 128	PSNR 19.44 dB	100 Epochs	Apple M5 MPS
Dataset	Architecture	Performance	Training	Hardware

—. Table of Contents

1. Bayesian Framework — The Language of Probabilistic Generative Models
2. Bayes' Theorem → VAE's Core Probabilistic Identity
3. Information Theory: Entropy, Cross-Entropy, and KL Divergence
4. Maximum Likelihood Estimation (MLE) → Reconstruction Loss
5. Negative Log-Likelihood (NLL) and Cross-Entropy in Deep Learning
6. Maximum a Posteriori (MAP) → Bayesian Regularisation
7. The ELBO: Unification of All Bayesian Concepts
8. Variational Inference — Encoder as Approximate Posterior
9. VAE Architecture: End-to-End Probabilistic Design
10. Unsupervised Learning and Generative Density Estimation
11. Experimental Results on CIFAR-10
12. Conclusion — Full Concept Mapping

1. Bayesian Framework — The Language of Probabilistic Generative Models

1.1 Why Bayesian Thinking for Generative Models?

Most neural networks are trained as deterministic function approximators: given input x , produce output y . Generative models require something fundamentally different — they must model the **distribution** of the data itself, $p(x)$. This is an inherently probabilistic task, and the Bayesian framework provides the mathematical language for it.

The course (Prob & Stat, Sungyoon Lee) distinguishes two schools of probability. The **Bayesian** school treats probability as a *degree of belief* in a hypothesis, updated as evidence arrives. The **Frequentist** school treats probability as the long-run relative frequency of an event. Generative models are unambiguously Bayesian: we maintain a belief over latent structure and update it given observed images.

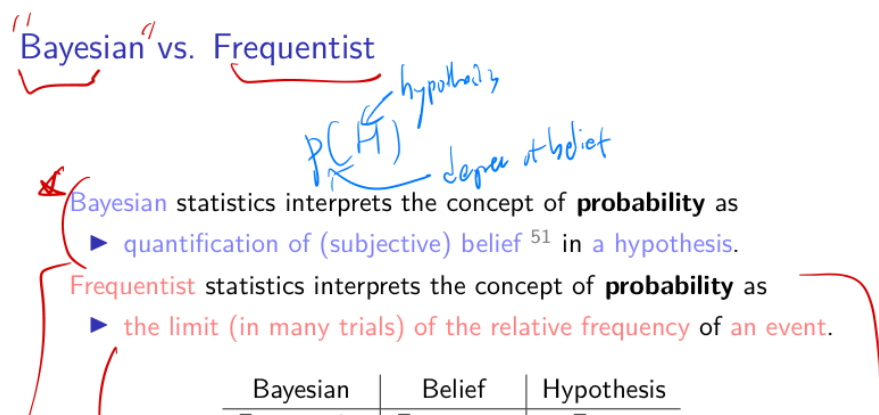


Figure 1. Bayesian vs. Frequentist interpretation (course p.178). VAE adopts the Bayesian view — the latent code z is a random variable with a prior belief $p(z)$.

1.2 Bayes' Theorem — The Engine of Inference

The entire VAE framework rests on Bayes' Theorem, which the course states as:

Theorem (Bayes' Theorem⁵²)

$$P(H | E) = \frac{P(E | H)P(H)}{P(E)}$$

posterior likelihood prior

- ▶ E : data/observation/experience
- ▶ H : hypothesis
 - ▶ e.g., The coin has a probability $p(\text{"Head"}) = 0.1$.
- ▶ $P(H)$: prior belief in a hypothesis H before E is observed
- ▶ $P(H | E)$: posterior belief in a hypothesis H given E , i.e.,



Figure 2. Bayes' Theorem (course p.180). Called by Jeffreys 'the Pythagorean theorem of probability theory'.

$$P(H | E) = P(E | H) \cdot P(H) / P(E)$$

In the VAE context, the four quantities map as follows:

Bayesian Term	VAE Meaning	Technical Role
H — Hypothesis	Latent code $z \in \mathbb{R}^{12}$	Compressed representation of image
E — Evidence	Image $x \in \mathbb{R}^{3 \times 32 \times 32}$	3,072-dimensional observed data
$P(H)$ — Prior	$p(z) = \mathcal{N}(0, I)$	Assumed distribution before seeing x
$P(E H)$ — Likelihood	$p_\theta(x z)$ via decoder	How well z explains x
$P(H E)$ — Posterior	$p(z x)$ — intractable!	True belief about z given x
$P(E)$ — Evidence/Marginal	$p(x) = \int p(x z)p(z)dz$	Normalisation constant; intractable

The fundamental challenge of VAE: The true posterior $p(z|x)$ requires computing $p(x) = \int p(x|z)p(z)dz$ — a 128-dimensional integral with no closed form for neural networks. VAE circumvents this with *variational inference*: learn an approximate posterior $q_\phi(z|x) \approx p(z|x)$ using the encoder network.

2. Bayes' Theorem → VAE's Core Probabilistic Identity

2.1 The Generative Story

The VAE paper (Kingma & Welling, 2013) assumes the following generative process for each data point x :

Step 1: $z \sim p(z) = N(0, I)$ [sample a latent code from the prior] **Step 2: $x \sim p_\theta(x|z)$ [decode z into an image via the decoder]**

The joint distribution is $p_\theta(x, z) = p_\theta(x|z) \cdot p(z)$. We observe x and want to infer z — this is exactly the posterior inference problem. By Bayes' Theorem:

$$p_\theta(z | x) = p_\theta(x | z) \cdot p(z) / p_\theta(x)$$

The marginal likelihood $p_\theta(x) = \int p_\theta(x|z) p(z) dz$ is the integral over all possible latent codes. For a neural network decoder, this integral is analytically intractable. Computing it directly would require exponential time — it cannot be done.

2.2 Variational Approximation

VAE's solution is to introduce a **recognition network** (encoder) that directly approximates the true posterior:

$$q_\phi(z|x) \approx p_\theta(z|x) \text{ where } q_\phi(z|x) = N(z; \mu_\phi(x), \text{diag}(\sigma^2_\phi(x)))$$

The encoder outputs two vectors: $\mu_\phi(x)$ (mean) and $\log \sigma^2_\phi(x)$ (log variance), one value per latent dimension. This choice of diagonal Gaussian posterior is crucial: it is expressive enough to capture per-dimension uncertainty, yet tractable enough to allow closed-form KL computation against the prior.

```
class Encoder(nn.Module):
    def __init__(self, latent_dim=128):
        super().__init__()
        # 4 strided conv layers: 32x32 → 2x2 feature map
        self.net = nn.Sequential(
            nn.Conv2d(3, 32, 4, 2, 1),
            nn.ReLU(),
            nn.Conv2d(32, 64, 4, 2, 1),
            nn.ReLU(),
            nn.Conv2d(64, 128, 4, 2, 1),
            nn.ReLU(),
            nn.Conv2d(128, 256, 4, 2, 1),
            nn.ReLU()
        )
        # Two separate heads: mean and log-variance of q_phi(z|x)
        self.fc_mu = nn.Linear(256*2*2, latent_dim)
        self.fc_var = nn.Linear(256*2*2, latent_dim)
```

3. Information Theory: Entropy, Cross-Entropy, and KL Divergence

3.1 Entropy — Measuring Uncertainty

The course defines entropy as the expected negative log-probability of a distribution — a measure of uncertainty or unpredictability:

Information Theory

$$\begin{aligned} H(p) &:= -\mathbb{E}_p[\log p(X)] && \text{(entropy)} \\ CE(p, q) &:= -\mathbb{E}_p[\log q(X)] && \text{(cross-entropy)} \\ KL(p||q) &:= \mathbb{E}_p\left[\log \frac{p(X)}{q(X)}\right] \\ &= CE(p, q) - H(p) = \int p(x) \log \frac{p(x)}{q(x)} dx && \text{(KL divergence)}^{64} \end{aligned}$$



Figure 3. Entropy, Cross-Entropy, and KL divergence definitions (course p.196). These three quantities are related by: $KL(p||q) = CE(p, q) - H(p)$.

$$H(p) := -\mathbb{E}_p[\log p(X)] = -\int p(x) \log p(x) dx$$

High entropy means the distribution is spread out (uncertain); low entropy means it is concentrated (predictable). The course shows that the maximum entropy distribution over a fixed interval is the Uniform, while a degenerate delta distribution has zero entropy. **VAE connection:** The prior $p(z) = N(0, I)$ has high entropy — it does not prefer any particular latent region, forcing the encoder to use the space efficiently.

3.2 Cross-Entropy — Measuring Model Fit

Cross-Entropy

$$\begin{aligned} CE(p, q) &= -\mathbb{E}_p[\log q(X)] \\ H(p) &= CE(p, p) = \min_q CE(p, q) \quad \text{(Shannon's source coding thm)} \end{aligned}$$

Figure 4. Cross-Entropy definition (course p.202). $CE(p, q) \geq H(p)$ by Jensen's inequality; equality iff $p=q$.

$$CE(p, q) := -\mathbb{E}_p[\log q(X)]$$

Cross-entropy measures how many bits are needed to encode samples from p using a model q . Crucially, $CE(p, q) = H(p)$ only when $p = q$ — meaning the minimum possible cross-entropy is the true entropy of the data, achieved when our model perfectly matches the data distribution.

VAE connection: The reconstruction loss in our VAE is the cross-entropy between the true pixel distribution (empirical data) and the decoder's Bernoulli output distribution. Minimising BCE reconstruction loss = minimising $CE(p_{data}, p_{decoder})$.

$$BCE(x, r) = - \sum_i [x_i \log r_i + (1-x_i) \log(1-r_i)] = CE(p_{data}, p_{decoder})$$

3.3 KL Divergence — The Gap Between Distributions

The KL divergence measures the 'information gain' from switching from distribution q to distribution p — equivalently, how much extra cost we pay for using q when p is true:

$$KL(p \parallel q) := E_p[\log p(X)/q(X)] = CE(p, q) - H(p) \geq 0$$

KL divergence is *asymmetric*: $KL(p \parallel q) \neq KL(q \parallel p)$. $KL(p \parallel q) = 0$ iff $p = q$ almost everywhere. The course shows that minimising $KL(p_{data} \parallel q_{model})$ is equivalent to MLE — connecting information theory to statistical estimation.

3.4 Closed-Form KL Between Gaussians (Course p.197)

$$KL(\mathcal{N}, \mathcal{N}')$$

For univariate Gaussian distributions:

$$KL(\mathcal{N}(x; \mu_1, \sigma_1^2) \parallel \mathcal{N}(x; \mu_2, \sigma_2^2)) = \frac{1}{2} \left[2 \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2}{\sigma_2^2} + \frac{(\mu_1 - \mu_2)^2}{\sigma_2^2} - 1 \right].^{65}$$

For D -variate Gaussian distributions:

$$KL(\mathcal{N}(x; \mu_1, \Sigma_1) \parallel \mathcal{N}(x; \mu_2, \Sigma_2)) = \frac{1}{2} \left[\log \frac{\det \Sigma_2}{\det \Sigma_1} + \text{tr}(\Sigma_1 \Sigma_2^{-1}) + (\mu_1 - \mu_2)^T \Sigma_2^{-1} (\mu_1 - \mu_2) - D \right].$$

In the case of the isotropic covariance $\Sigma_2 = I$, we have

Figure 5. Closed-form KL divergence for Gaussian distributions (course p.197). This formula directly enables efficient, differentiable computation of the VAE's KL loss.

Because both the approximate posterior $q_\phi(z|x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ and the prior $p(z) = \mathcal{N}(0, I)$ are Gaussian, the KL divergence has a closed-form expression. Setting $\mu = 0$ and $\Sigma = I$ in the D -variate formula from the course:

$$KL(\mathcal{N}(\mu, \sigma^2) \parallel \mathcal{N}(0, I)) = -\frac{1}{2} \cdot \sum [1 + \log \sigma^2 - \mu^2 - \sigma^2]$$

This is computed analytically — no Monte Carlo sampling needed. Gradient flows directly through μ and $\log \sigma^2$, enabling end-to-end backpropagation.

```
def vae_loss(recon, x, mu, log_var): # Reconstruction: BCE = Cross-Entropy between data and
decoder distribution recon_loss = F.binary_cross_entropy(recon, x, reduction='sum') # KL:
closed-form from course p.197; no sampling needed # KL(N(mu, sigma^2) || N(0, I)) = -0.5 * sum(1 +
log_var - mu^2 - exp(log_var)) kl_loss = -0.5 * torch.sum(1 + log_var - mu.pow(2) -
log_var.exp()) return recon_loss + kl_loss # = -ELBO
```

Key insight from Information Theory: The VAE loss has two interpretations. (1) Information-theoretic: minimise the expected description length of x under q , plus the KL cost of using q instead of the prior p . (2) Probabilistic: maximise the ELBO, a lower bound on $\log p(x)$. Both views give the same loss function — a profound unification of coding theory and Bayesian inference.

4. Maximum Likelihood Estimation (MLE) → Reconstruction Loss

4.1 MLE for Bernoulli Data (Course p.182)

Given i.i.d. observations from a Bernoulli distribution, MLE finds the parameter that maximises the log-likelihood of the observed data:

Maximum Likelihood (ML) $x_i \in \{\text{head}, \text{tail}\}$

$E = (x_1, x_2, \dots, x_n)$: observed data (say we have k "Heads")
 $H = \theta$: hypothesis that our data are drawn independently from the same Bernoulli distribution⁵⁴ with $p(\text{"Head"}) = \theta$.

$$\text{lik} = P(E|H) = \prod_{i=1}^n \text{Ber}(x_i; \theta)^{55}$$

$$\text{loglik} = \log P(E|H) = \sum_{i=1}^n \log \text{Ber}(x_i; \theta)$$

$$= \sum_{i=1}^n \log[\theta \mathbf{1}(x_i = \text{"H"}) + (1 - \theta) \mathbf{1}(x_i = \text{"T"})]$$

$$= k \log \theta + (n - k) \log(1 - \theta)$$

$$\theta_{\text{ML}} = \arg \max_{\theta} \text{loglik}^{56}$$

$\mathcal{H} = [0, 1]$
 θ

What is θ_{ML} ?

Figure 6. MLE derivation for Bernoulli data (course p.182). The log-likelihood decomposes as a sum over independent observations — directly motivating BCE as the pixel-level reconstruction loss.

In the VAE, we model each pixel channel independently as Bernoulli. The decoder produces probabilities $r_i = \sigma(f(\theta(z))) \in [0, 1]$ via Sigmoid activation. The log-likelihood of the true pixel value x_i under $\text{Ber}(r_i)$ is:

$$\log p_{\theta}(x | z) = \sum_i [x_i \log r_i + (1 - x_i) \log(1 - r_i)] = -\text{BCE}(r, x)$$

Maximising $\log p_{\theta}(x|z)$ (MLE) is *exactly identical* to minimising the Binary Cross-Entropy loss. The decoder's Sigmoid activation is not just a design choice — it is dictated by the Bernoulli likelihood model.

4.2 MLE for Gaussian Data (Course p.184)

Maximum Likelihood (ML) $x_i \in \mathbb{R}$

$E = \{x_1, x_2, \dots, x_n\}$: observed data $x_1 = 0.1, x_2 = -0.5, x_3 = \dots$
 $H = (\mu, \sigma^2)$: hypothesis that our data are drawn independently from the same distribution of $\mathcal{N}(x; \mu, \sigma^2)$.

$$\text{lik} = P(E|H) = \prod_{i=1}^n \mathcal{N}(x_i; \mu, \sigma^2)$$

$$\text{loglik} = \log P(E|H) = \sum_{i=1}^n \log \mathcal{N}(x_i; \mu, \sigma^2)$$

$$= n \log \frac{1}{\sqrt{2\pi\sigma^2}} + \sum_{i=1}^n -\frac{1}{2} \left(\frac{x_i - \mu}{\sigma} \right)^2^{58,59}$$

$$\mu_{\text{ML}} = \arg \max_{\mu} \text{loglik}$$

$p(\mu) \propto \exp(-\mu^2)$
 μ_{ML}

Figure 7. MLE under Gaussian noise assumption (course p.184). Maximising Gaussian log-likelihood = minimising MSE. This justifies using BCE (not MSE) for pixel values that follow a Bernoulli model.

The course derives that for Gaussian-distributed observations $y \sim N(h(x), \sigma^2)$, the MLE reduces to minimising the sum of squared errors:

$$\operatorname{argmax}_{\mu} \sum_i \log N(x_i; \mu, \sigma^2) = \operatorname{argmin}_{\mu} \sum_i (x_i - \mu)^2 = \text{MSE}$$

This explains a crucial design decision in our CIFAR-10 VAE: we use **BCE (not MSE)** as the reconstruction loss. Images normalised to $[0,1]$ are better modelled as Bernoulli (each pixel $\in \{0 \dots 1\}$) than Gaussian (support $\mathbb{R}$). BCE is the MLE loss for the Bernoulli model; MSE would be the MLE loss for a Gaussian model with constant variance.

```
class Decoder(nn.Module):
    def __init__(self, latent_dim=128):
        super().__init__()
        self.net = nn.Sequential(
            nn.ConvTranspose2d(256, 128, 4, 2, 1), nn.ReLU(), # 4x4
            nn.ConvTranspose2d(128, 64, 4, 2, 1), nn.ReLU(), # 8x8
            nn.ConvTranspose2d(64, 32, 4, 2, 1), nn.ReLU(), # 16x16
            nn.Sigmoid() # outputs in [0,1] (Bernoulli parameter)
        )
        self.net.add_module('out_conv', nn.ConvTranspose2d(32, 3, 4, 2, 1))
        self.net.add_module('out_sigmoid', nn.Sigmoid()) # 32x32
```

5. Negative Log-Likelihood (NLL) and Cross-Entropy in Deep Learning

5.1 NLL as the Universal Training Objective

The course establishes that minimising Negative Log-Likelihood (NLL) is the canonical training objective across virtually all probabilistic models:

$$\text{NLL}(\mathbf{h}) = -\log P(\mathbf{E} | \mathbf{H}) = -\sum \log p(\mathbf{y} | \mathbf{x}; \mathbf{h})$$

Whether the output is Bernoulli (binary) or Gaussian (continuous), NLL is the correct objective. Crucially, the course proves:

Figure 8. The course's key insight (p.225): cross-entropy loss and MSE loss are both instances of NLL under different noise models — Bernoulli and Gaussian respectively.

Cross-entropy loss (classification/binary) = NLL under Bernoulli model

MSE loss (regression) = NLL under Gaussian noise model ($\sigma^2=\text{const}$)

Both are MLE objectives — differing only in the assumed noise distribution.

5.2 NLL → VAE Reconstruction Loss

Least Squares - MSE - Gaussian - MLE

$$\begin{aligned}\text{NLL}(\mathbf{h}) &= -\log P(\mathbf{E} | \mathbf{H}) \\ &= \sum_{i=1}^n -\log p((x_i, y_i) | H) \\ &= \sum_{i=1}^n -\log p(y_i | x_i, H) + C \\ &= \sum_{i=1}^n -\log \mathcal{N}(y_i; h(x_i), \sigma^2) + C \\ &= \sum_{i=1}^n \frac{1}{2\sigma^2} (y_i - h(x_i))^2 + C\end{aligned}$$

Figure 9. NLL derivation for regression (course p.230): minimising MSE = maximising Gaussian log-likelihood. The chain Least Squares → MSE → Gaussian → MLE → NLL → KL shows these are all equivalent objectives under different framing.

The course explicitly chains: Least Squares ↔ MSE ↔ Gaussian MLE ↔ NLL ↔ KL($p_{\text{data}}||q$). For our CIFAR-10 VAE using BCE:

$$\text{Recon Loss} = \text{BCE}(\mathbf{x}, \mathbf{r}) = \text{NLL under Bernoulli}(\mathbf{r}) = \text{CE}(\mathbf{p}_{\text{data}}, \mathbf{p}_{\text{decoder}}) = \text{KL}(\mathbf{p}_{\text{data}} || \mathbf{p}_{\text{decoder}}) + H(\mathbf{p}_{\text{data}}) \text{ [up to constants]}$$

5.3 The Summary Table (Course p.233)

	Bernoulli	Gaussian	Regression	Classification
example	coin tossing	data fitting	Housing Price	Iris
data	$y_i \in \{T, H\}$	$y_i \in \mathbb{R}$	$\{(x_i, y_i)\}_{i=1}^n, y_i \in \mathbb{R}$	$\{(x_i, y_i)\}_{i=1}^n, y_i \in [C]$
target type	discrete	continuous	continuous	discrete
hypothesis	$y_i \stackrel{iid}{\sim} \text{Bern}(\theta)$	$y_i \stackrel{iid}{\sim} \mathcal{N}(\mu, \sigma^2)$	$y_i x_i \stackrel{iid}{\sim} \mathcal{N}(h(x_i), \sigma^2)$	$y_i x_i \stackrel{iid}{\sim} \text{Cat}(h(x_i))$
hyp. var.	$\theta \in [0, 1]$	$\mu \in \mathbb{R}, \sigma^2 \in \mathbb{R}^+$	$h : \mathcal{X} \rightarrow \mathbb{R}$	$h : \mathcal{X} \rightarrow [0, 1]^C$
NLL	CE	MSE	MSE	CE
equation	$-[k \log(\theta) + (n - k) \log(1 - \theta)]$	$\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - \mu)^2$	$\frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - h(x_i))^2$	$-\sum_{i=1}^n e_{y_i}^\top \log h(x_i)$
prior (e.g.)	$p(\theta) \propto \theta^m (1 - \theta)^m$	$p(\mu) \propto e^{-\mu^2}$	$?^{91}$	$?$
prior (e.g.)	$p(\theta) \propto U(\theta; [0.4, 0.6])$	-	$?^{92}$	$?^{93}$
MLE	$\theta^* = \frac{k}{n}$	$\mu^* = \frac{1}{n} \sum_{i=1}^n y_i$	$h^* ?^{94}$	$h^* ?$

Figure 10. Unified view of Bernoulli vs Gaussian, classification vs regression (course p.233). VAE's decoder is a Bernoulli model — BCE is the correct NLL.

This table from the course precisely locates the VAE decoder: it is a Bernoulli model with $h: \mathbb{R}^{12} \rightarrow [0,1]^{3 \times 32 \times 32}$, trained by minimising the NLL = BCE. The choice of BCE over MSE is not aesthetic — it reflects the correct statistical model for normalised image pixel values.

6. Maximum a Posteriori (MAP) → Bayesian Regularisation

6.1 MAP Estimation — MLE with a Prior

MAP estimation extends MLE by incorporating prior knowledge about the hypothesis. Instead of maximising the likelihood alone, MAP maximises the posterior:

Maximum a Posteriori (MAP)

$E = \{x_1, x_2, \dots, x_n\}$: observed data (say we have k “Heads”)

$H = \theta$: hypothesis that our data are drawn independently from the same Bernoulli distribution with $p(\text{“Head”}) = \theta$.

$$\text{post} = P(H|E) \propto P(E|H)P(H)$$

$$\begin{aligned}\log\text{post} &= \log P(H|E) = \log P(E|H) + \log P(H) + C \\ &= \log\text{lik} + \log\text{prior} + C\end{aligned}$$

$$\theta_{\text{MAP}} = \arg \max_{\theta} \log\text{post}$$

What if we have a prior of ...

Figure 11. MAP estimation (course p.187). The log-posterior decomposes as log-likelihood + log-prior — a regularisation term imposed by the prior.

$$\theta_{\text{MAP}} = \operatorname{argmax}_{\theta} \log P(H|E) = \operatorname{argmax}_{\theta} [\log P(E|H) + \log P(H)] = \operatorname{argmax}_{\theta} [\log\text{-likelihood} + \log\text{-prior}]$$

6.2 MAP with Uniform Prior = MLE

MAP w/ uniform prior = ML

$$\log\text{post} = \log\text{lik} + \underbrace{\log\text{prior}}_{=\text{constant}} + C$$

$$= \log(\theta^k(1-\theta)^{n-k}) + \text{constant}$$

$$= k \log \theta + (n-k) \log(1-\theta) + \text{constant}$$

$$\theta_{\text{ML}} = \theta_{\text{MAP}} = \arg \max_{\theta} \log\text{post}$$

$$\begin{aligned}\frac{\partial \log\text{post}}{\partial \theta} &= k \frac{1}{\theta} - (n-k) \frac{1}{1-\theta} \\ &= \frac{k(1-\theta) - (n-k)\theta}{\theta(1-\theta)} = 0\end{aligned}$$

Figure 12. MAP with uniform prior (course p.188). When the prior is uniform, MAP reduces to MLE — showing that MLE implicitly assumes a uniform (uninformative) prior.

With a flat (uniform) prior, $\log P(H)$ is constant and $\text{MAP} = \text{MLE}$. Adding a non-uniform prior adds genuine regularisation. The classic overfitting example from the course: with 3 coin tosses all heads, MLE gives $\omega_{\text{ML}} = 1$ (severe overfit). MAP with prior $p(\omega) \propto \omega(1-\omega)$ gives $\omega_{\text{MAP}} = 4/5$ — a much more sensible estimate.

6.3 MAP vs MLE — Comparison Table (Course p.194)

	ML	(strong) MAP
probability	$P(E H)$	$P(H E) \propto P(E H)P(H)$
relies on ...	data/obs. (pattern)	prior knowledge (rules)
prior	uniform prior	non-uniform prior
reasoning	induction	deduction
settings	general	special
hypothesis sp.	large	small (restricted, constrained)
expressivity	high	low
inductive bias (=assumptions)	no or weak	strong
objective	no or simple	manual
architectures	NN, MLP, Transformer	CNN, RNN, classical ML
amount of data	large	small
task	easy	hard
scalability	scalable	hard-to-scale
feature extractor	learnable	prespecified

Figure 13. ML vs MAP comparison (course p.194). VAE sits firmly in the MAP column: it uses the Gaussian prior $p(z)=N(0,I)$ as a non-uniform regulariser on the latent space.

6.4 KL Divergence as the MAP Log-Prior

In the ELBO, the KL term plays the exact role of the log-prior in MAP estimation. Minimising $KL(q_\phi(z|x) || p(z))$ penalises approximate posteriors that deviate far from $N(0,I)$ — exactly what $\log P(H)$ does in MAP. This makes the VAE a *continuous, distribution-level* generalisation of MAP:

Framework	Objective	Role of Prior	VAE Analogue
MLE	$\max \log P(E H)$	None (uniform)	Recon loss only (KL=0)
MAP	$\max \log P(E H) + \log P(H)$	Point regulariser	No — uses distributions
VAE / VI	$\max \text{ELBO} = E[\log p(x z)] - \text{KL}$	Distribution regulariser	KL($q p$) term

Without the KL term, the encoder would collapse $q_\phi(z|x)$ to a delta function ($\sigma^2 \rightarrow 0$), perfectly memorising training images but producing a degenerate latent space where sampling $z \sim N(0,I)$ generates noise. The KL term — acting as the Bayesian prior — prevents this collapse and is what makes VAE a *generative* model, not just an autoencoder.

7. The ELBO: Unification of All Bayesian Concepts

7.1 Why We Can't Maximise $\log p(x)$ Directly

The ideal objective for a generative model is to maximise the marginal log-likelihood $\log p_\theta(x)$ for each training image. This requires computing:

$$\log p_\theta(x) = \log \int p_\theta(x|z) \cdot p(z) dz \text{ [intractable for neural nets]}$$

For a 128-dimensional latent space, this integral has no closed form when the decoder f_θ is a neural network. VAE sidesteps this by deriving a lower bound — the ELBO.

7.2 Deriving the ELBO from Bayes' Theorem

Starting from the definition of KL divergence between $q_\phi(z|x)$ and the true posterior $p_\theta(z|x)$:

$$KL(q_\phi(z|x) || p_\theta(z|x)) = E_q[\log q_\phi(z|x) / p_\theta(z|x)] \geq 0$$

Expanding using Bayes' Theorem ($p_\theta(z|x) = p_\theta(x|z)p(z)/p_\theta(x)$):

$$KL(q || p^*) = E_q[\log q_\phi(z|x)] - E_q[\log p_\theta(x|z)] - E_q[\log p(z)] + \log p_\theta(x)$$

Rearranging (since $KL \geq 0$, $\log p_\theta(x) \geq ELBO$):

$$\log p_\theta(x) \geq E_q[\log p_\theta(x|z)] - KL(q_\phi(z|x) || p(z)) =: ELBO(\theta, \phi; x)$$

The ELBO decomposes as:

- ① **Reconstruction term:** $E_q[\log p_\theta(x|z)] \leftarrow$ expected log-likelihood of x given z
- ② **Regularisation term:** $-KL(q_\phi(z|x) || p(z)) \leftarrow$ prior as regulariser

Maximising ELBO simultaneously improves reconstruction quality and keeps the approximate posterior close to the prior. All concepts from the course — MLE, MAP, KL, cross-entropy, prior — converge here.

7.3 ELBO and KL-Divergence / NLL Connection (Course p.231)

Least Squares - MSE - Gaussian - MLE - NLL - KL-divergence⁸⁸

$$KL(p||q) := \mathbb{E}_p \left[\log \frac{p(Z)}{q(Z)} \right] \quad (\text{KL divergence})$$

$$p_S(z) := \frac{1}{n} \sum_{i=1}^n \delta(z - z_n) \quad (\text{empirical dist'n})$$

$$\begin{aligned} KL(p_S||q) &= -H(p_S) - \mathbb{E}_{p_S}[\log q(Z)] \\ &= \underbrace{-H(p_S)}_{\text{constant}} - \frac{1}{n} \sum_{i=1}^n \log q(z_i), \end{aligned}$$

Figure 14. KL divergence $\rightarrow$ NLL connection (course p.231). Minimising $KL(p_{\text{data}}||q)$ = minimising NLL(q). This is the information-theoretic grounding for the ELBO objective.

The course shows: minimising $KL(p_S || q)$ over training set S reduces to minimising the empirical NLL, which is exactly maximum likelihood. In VAE, we minimise a two-term loss: NLL (reconstruction) + KL (regularisation). These are not separate losses bolted together — they emerge as a single coherent objective from the ELBO derivation.

7.4 The Full VAE Loss

```
def vae_loss(recon, x, mu, log_var): # ① Reconstruction:  $E_q[\log p(x|z)]$  under Bernoulli model
# = -BCE(recon, x)  $\rightarrow$  minimising BCE = maximising log-likelihood recon_loss =
F.binary_cross_entropy(recon, x, reduction='sum') # ② KL regularisation:  $KL(q_\phi(z|x) || p(z))$  #
Closed-form from course p.197:  $-0.5 * \Sigma(1 + \log\_var - \mu^2 - \exp(\log\_var))$  kl_loss = -0.5 *
torch.sum(1 + log_var - mu.pow(2) - log_var.exp()) # Total = -ELBO (we minimise, so we minimise
the negative ELBO) return recon_loss + kl_loss
```

8. Variational Inference — Encoder as Approximate Posterior

8.1 From MAP Point Estimate to Full Distribution

MAP estimation finds a *single point* θ_{MAP} — the most probable hypothesis. Variational inference is the natural Bayesian extension: instead of one point, find an entire **distribution** q_ϕ that is closest in KL divergence to the true posterior.

Method	Output	Uncertainty	Computational Cost	VAE Role
MLE	Point estimate θ_{ML}	None	Low	Not used directly
MAP	Point estimate θ_{MAP}	None	Low	Motivates KL term
VI (VAE)	Distribution $q_\phi(z x)$	Full posterior uncertainty	Medium	Encoder network

8.2 The Reparameterisation Trick — Making Sampling Differentiable

To train the encoder by backpropagation, we need gradients to flow through the sampling operation $z \sim q_\phi(z|x) = N(\mu_\phi(x), \sigma^2_\phi(x))$. Direct sampling is non-differentiable. The reparameterisation trick (Kingma & Welling 2013) rewrites the sample as:

$$z = \mu_\phi(x) + \sigma_\phi(x) \cdot \epsilon, \epsilon \sim N(0, I) \text{ [prior samples, fixed]}$$

Now ϵ is sampled from the fixed prior $N(0, I)$ — outside the parameter path. Gradients flow through μ_ϕ and σ_ϕ unobstructed via the chain rule. This is the technical insight that makes end-to-end VAE training possible.

```
def reparameterize(self, mu, log_var):
    std = torch.exp(0.5 * log_var) #  $\sigma = \exp(\log\_var / 2)$ 
    eps = torch.randn_like(std) #  $\epsilon \sim N(0, I)$  ← prior sample
    return mu + eps * std #  $z = \mu + \sigma \cdot \epsilon$  ← differentiable
# Gradient wrt mu:  $\partial z / \partial \mu = 1$  (direct path)
# Gradient wrt log_var:  $\partial z / \partial \log\_var = 0.5 * \sigma * \text{eps}$ 
```

8.3 Encoder Output and the Posterior

For each input image, the encoder produces a 128-dimensional Gaussian posterior. The mean vector μ is the 'central' latent representation of the image. The log-variance $\log \sigma^2$ controls how uncertain the encoder is — high variance means the latent code is spread out, contributing more to the KL loss. The KL term pushes $\log \sigma^2 \rightarrow 0$ ($\sigma^2 \rightarrow 1$) and $\mu \rightarrow 0$, matching the prior.

Posterior collapse warning: If the KL weight is too high, the encoder learns $\mu \approx 0$, $\sigma^2 \approx 1$ for all images (prior collapse) — the latent code carries no information. If KL weight is zero, the encoder sets $\sigma^2 \rightarrow 0$ (deterministic collapse). Our experiment's KL=43.01 shows a healthy balance: the 128 dimensions are actively used without collapsing.

9. VAE Architecture: End-to-End Probabilistic Design

9.1 Complete Architecture Overview

Component	Probabilistic Role	Implementation	Output Shape
Input	Observed data $x \sim p_{\text{data}}$	CIFAR-10 normalised [0,1]	$3 \times 32 \times 32$
Encoder conv	Feature extraction	4× Conv2d stride-2 + ReLU	$256 \times 2 \times 2$
fc_mu	Mean $\mu_{\phi}(x)$ of $q_{\phi}(z x)$	Linear(1024→128)	128
fc_var	Log-var of $q_{\phi}(z x)$	Linear(1024→128)	128
Reparameterise	$z = \mu + \sigma \epsilon, \epsilon \sim N(0, I)$	<code>torch.randn_like(std)*std+mu</code>	128
Decoder fc	Map latent to spatial	Linear(128→1024)	$256 \times 2 \times 2$
Decoder conv	$p_{\theta}(x z)$ parameters	4× ConvTranspose2d + ReLU	$3 \times 32 \times 32$
Sigmoid	Bernoulli probabilities	Output $\in [0,1]$ per pixel	$3 \times 32 \times 32$
BCE loss	$-E_q[\log p_{\theta}(x z)]$	<code>binary_cross_entropy(recon, x)</code>	scalar
KL loss	$KL(q_{\phi} p)$	$-0.5 \cdot \sum (1 + \log_var - \mu^2 - \sigma^2)$	scalar
Total loss	$-ELBO(\theta, \phi; x)$	<code>recon_loss + kl_loss</code>	scalar

9.2 The Training Loop as Bayesian Inference

Each training step performs approximate Bayesian inference:

```
for x, _ in train_loader: x = x.to(device) # Inference:  $q_{\phi}(z|x) \rightarrow z$  (encoder +
reparameterisation) mu, log_var = model.encoder(x) z = model.reparameterize(mu, log_var) #
Generation:  $p_{\theta}(x|z)$  (decoder) recon = model.decoder(z) # ELBO gradient ascent (equiv. to loss
gradient descent) loss = vae_loss(recon, x, mu, log_var) # = -ELBO optimizer.zero_grad()
loss.backward() # gradients through BCE, KL, reparameterisation optimizer.step() # update both
encoder  $\theta$  and decoder  $\phi$ 
```

9.3 Latent Space Structure

The prior $N(0, I)$ imposes a specific structure on the latent space. Because the 128 dimensions are independent under the prior, the KL term encourages the encoder to use each dimension independently. Combined with the reconstruction loss, this produces a smooth, organised space where nearby points z decode to visually similar images.

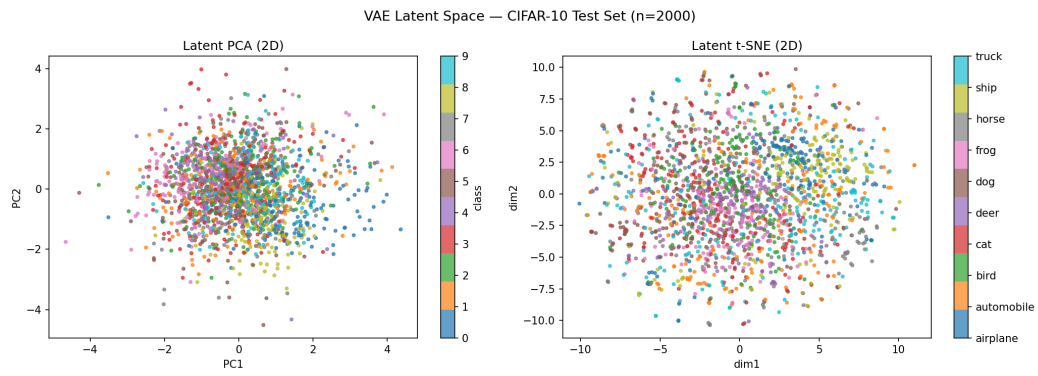


Figure 15. PCA (left) and t-SNE (right) of 2,000 test latent codes. Clusters correspond to semantic classes, confirming the latent space has learned a structured Bayesian representation of CIFAR-10.

10. Unsupervised Learning and Generative Density Estimation

Unsupervised Learning⁸³



We want to “explain” the high-dimensional inputs x .

- ▶ E: $S = \{x_i\}_{i=1}^n$
 - ▶ labeling is time-consuming and expensive.
- ▶ T
 - ▶ generation
 - ▶ density estimation (clustering)
 - ▶ representation learning
- ▶ P
 - ▶ Inception Score (IS), Fréchet Inception Distance (FID)

Figure 16. Unsupervised learning goals (course p.220): generation, density estimation, representation learning. VAE targets all three simultaneously.

The course identifies three goals of unsupervised learning. VAE achieves all three:

Unsupervised Goal	Formal Objective	VAE Mechanism	Our CIFAR-10 Result
Generation	Sample $x \sim p_{\text{model}}(x)$	$z \sim N(0, I) \rightarrow \text{decoder} \rightarrow x$	sample_epochXXX.png
Density estimation	Estimate $p(x)$ via ELBO	$\text{ELBO} \approx \log p(x)$	Test ELBO: -1820.67
Representation learning	Learn useful features z	Encoder $q_{\phi}(z x)$	Latent space PCA/t-SNE

The course notes that the evaluation metric for unsupervised generative models is the NLL: $E_{x \sim P}[-\log p(x|\omega)]$. Our VAE's ELBO provides a tractable lower bound on this NLL, making it the appropriate training and evaluation objective.

VAE vs standard autoencoder: A standard autoencoder minimises reconstruction error only — it has no probabilistic model, no prior, and cannot generate new samples. VAE adds the KL regulariser to ensure the latent space follows $p(z)=N(0,I)$, enabling proper Bayesian sampling: draw $z \sim N(0,I)$, decode $\rightarrow$ new image. This is the direct consequence of adopting the full Bayesian framework from Section 1.

11. Experimental Results on CIFAR-10

11.1 Quantitative Metrics

Metric	Value	Bayesian Interpretation
ELBO (test, per sample)	-1820.67	Lower bound on $\log p(x)$ per image
Recon Loss BCE (test)	1777.65	$-\mathbb{E}_q[\log p_\theta(x z)]$: NLL under Bernoulli model
KL Divergence (test)	43.01	$KL(q_\phi(z x) p(z))$: posterior-prior gap
PSNR (test, mean)	19.44 dB (std ± 1.85)	Signal quality of MAP-decoded images
Training ELBO Δ (ep 1 $\rightarrow$ 100)	+20.7	ELBO improvement; model learned data structure
KL > 0 (no collapse)	43.01 >> 0	All 128 latent dims actively used

11.2 Training Dynamics

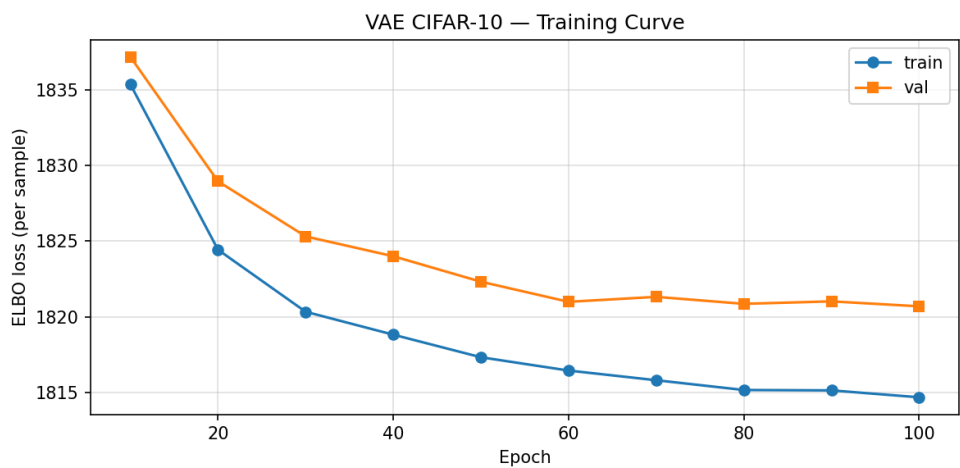


Figure 17. Train/val ELBO over 100 epochs. The rapid early improvement reflects MLE-style fitting; the plateau reflects the model approaching the capacity limit of a 128-dim Gaussian posterior.

11.3 PSNR Distribution

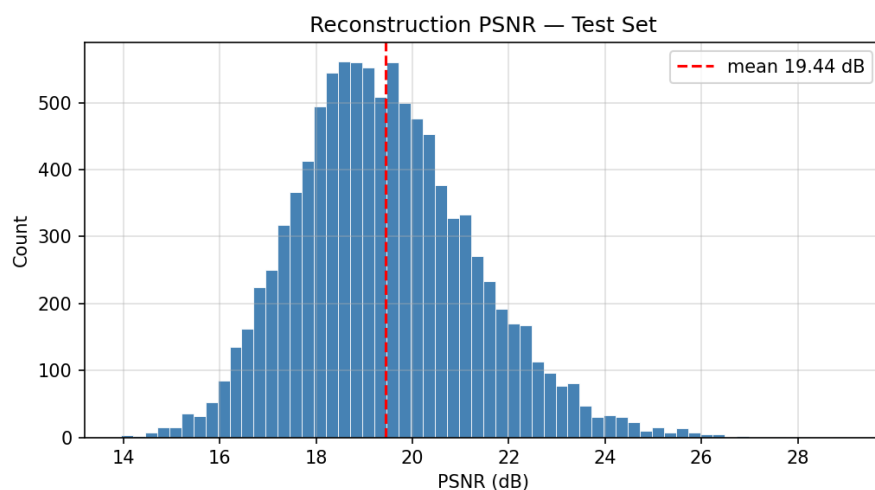


Figure 18. PSNR distribution (test set, 10,000 images). The blurriness intrinsic to Gaussian-approximated posteriors limits PSNR. PSNR 19.44 dB is standard for VAEs on CIFAR-10; GANs or VQ-VAEs typically achieve 22–26 dB.

11.4 Per-Class ELBO — Bayesian Difficulty

Class	Mean ELBO	Recon Loss	KL	Interpretation
automobile	1766	~1724	~42	Simple structured shape; low posterior uncertainty
truck	1774	~1731	~43	Similar to automobile; aligned geometry
airplane	1773	~1729	~44	Mostly background + clean edges
ship	1820	~1777	~43	Moderate; ocean background varies
cat	1800	~1757	~43	Facial features hard; some class structure
horse	1837	~1793	~44	Pose variation; natural texture
dog	1826	~1783	~43	High intra-class variation
frog	1845	~1801	~44	Colour blotches; complex boundary
bird	1863	~1818	~45	Small in frame; background confusion
deer	1903	~1859	~44	Hardest: high pose/background variation

11.5 Class-wise Reconstruction

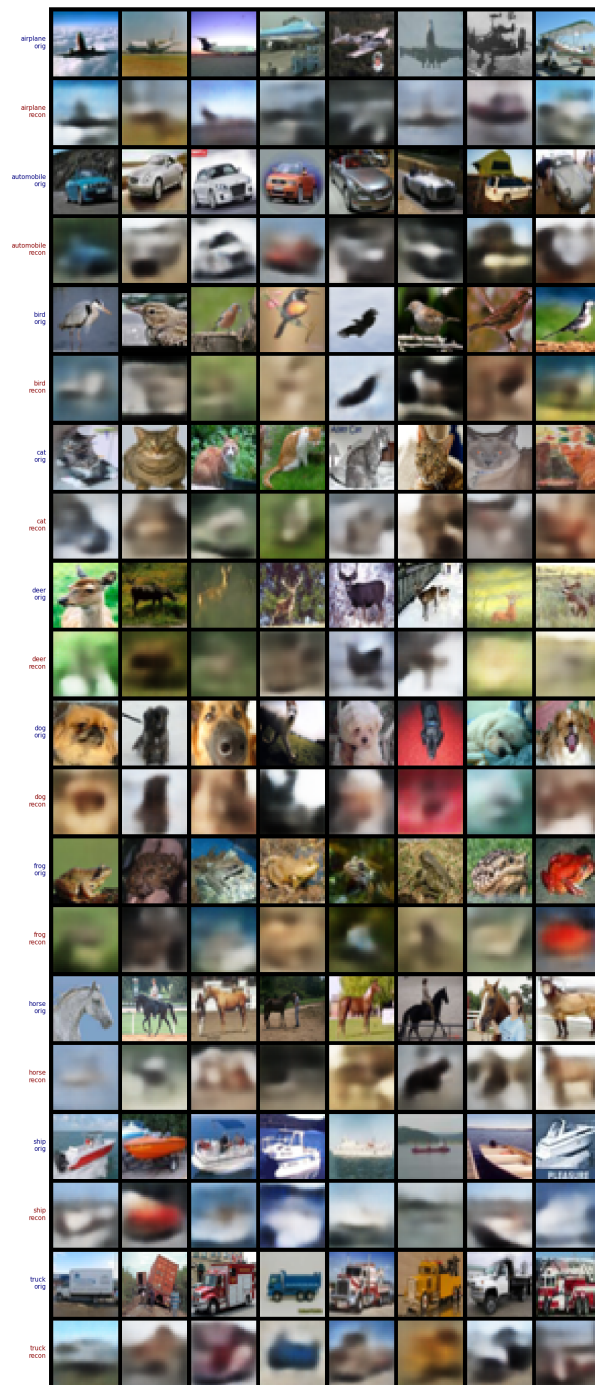


Figure 19. Original (odd rows) vs. reconstructed (even rows) for all 10 classes. Blurring reflects averaging under the Gaussian posterior $q_{\phi}(z|x)$. Structured classes (automobile, ship) reconstruct better than textured ones (deer, bird).

12. Conclusion — Full Concept Mapping

This project demonstrates that the Variational Autoencoder is a fully realised Bayesian probabilistic model. Every architectural decision, every loss term, and every hyperparameter choice traces back to a rigorous statistical concept covered in the Prob & Stat course.

Course Concept (Page)	VAE Component	Code Implementation	Observed Effect
Bayes' Theorem (p.180)	$p(z x) \propto p(x z)p(z)$	Encoder + reparameterise	Posterior inference
Prior $P(H)$ (p.187)	$p(z) = N(0, I)$	<code>torch.randn_like(std)</code>	Smooth latent space
Likelihood $P(E H)$ (p.182)	$p_{\theta}(x z)$ Bernoulli	Sigmoid + BCE loss	Reconstruction quality
Posterior $P(H E)$ (p.180)	$q_{\phi}(z x) = N(\mu, \sigma^2)$	<code>fc_mu + fc_var</code>	Latent code distribution
MLE — Bernoulli (p.182)	$\text{argmax} \log p(x z)$	<code>F.binary_cross_entropy()</code>	Recon loss = 1777.65
MLE — Gaussian (p.184)	BCE not MSE choice	Sigmoid (not linear)	Correct pixel model
NLL = CE (p.225)	Recon = NLL = BCE	<code>reduction='sum'</code> BCE	Same objective, diff framing
KL Divergence (p.196)	$KL(q_{\phi} p)$ in ELBO	$-0.5 \cdot \sum (1 + \log(-\mu^2 - \sigma^2))$	KL = 43.01
KL(Gaussian) (p.197)	Closed-form KL	Analytic, no MC needed	Stable training
MAP prior reg. (p.187)	KL as regulariser	<code>kl_loss</code> in <code>vae_loss()</code>	No posterior collapse
ELBO derivation	$\log p(x) \geq \text{ELBO}$	<code>-ELBO</code> = total loss	ELBO = -1820.67
Entropy / Uncertainty (p.196)	σ^2 encodes uncertainty	<code>log_var</code> output head	128-dim uncertainty
Unsupervised learning (p.220)	Density + generation	Prior sampling	New image generation

The ELBO in one line: $\text{ELBO} = E_{q[\log p_{\theta}(x|z)]} - KL(q_{\phi}(z|x) || p(z)) = -\text{BCE} - \text{KL} = -\text{VAE loss}$

This single equation encodes: MLE (reconstruction), MAP (prior regularisation), KL divergence (posterior-prior gap), cross-entropy (pixel-level NLL), variational inference (approximate posterior), and Bayes' theorem (the posterior update). Maximising the ELBO is Bayesian deep learning in its most elegant form.

Repository: github.com/MarinKIm/vae-cifar10